

7-Day TypeScript Mastery – Project-Based Plan

Day 1 – Core Basics

- What TypeScript is and why it matters
- Setup (tsc, tsconfig.json, integration with Node/React)
- Basic types: string, number, boolean, any, unknown, never, void
- Type inference & type annotations

Project: Create a simple CLI calculator (add, subtract, multiply, divide) using typed functions.

Day 2 – Functions & Objects

- Function return types, optional & default params
- Object typing, readonly, optional properties
- Arrays, Tuples
- Enums vs Literal types

Project: Build a 'User Profile Manager' that creates, updates, and displays typed user objects.

Day 3 – Interfaces & Type Aliases

- interface vs type
- Extending interfaces, intersection types (&)
- Union types (|)
- Narrowing with typeof, in, instanceof

Project: Create a 'Shape Drawer' app where shapes (circle, rectangle, triangle) implement an interface.

Day 4 – Advanced Types

- Generics ()
- Utility types: Partial, Pick, Omit, Record, Readonly, ReturnType
- Mapped types
- Conditional types (T extends U ? X : Y)

Project: Build a generic 'API Response Wrapper' and use utility types to transform data models.

Day 5 – Classes & OOP

- Classes, public/private/protected, readonly
- Implements (class implements interface)
- Abstract classes
- Getters/Setters

Project: Develop a 'Library System' with User and Admin classes, and a Book repository using generics.

Day 6 – TypeScript with Real Projects

- TypeScript + React: Props & State types
- TypeScript + Node/Express
- Type assertions & guards
- Module systems (ESM, CommonJS)

Project: Create a small CRUD Todo App: Backend with Express + TS, Frontend with React + TS.

Day 7 – Deep Dive & Patterns

- Discriminated Unions, Exhaustive checks with never
- Declaration merging
- Ambient declarations (.d.ts)
- Config best practices (strict, noImplicitAny)

Project: Refactor an existing JavaScript mini-project fully into TypeScript with strict configs enabled.